



DEPARTMENT OF ARTIFICIAL INTELLIGENCE — THIRD YEAR

NATURAL LANGUAGE PROCESSING

Word Representation

One-Hot Encoding, Word Embeddings & Word2Vec

LECTURE 4

Learning Outcomes

By the end of this lecture, students will be able to:

01. Explain the limitations of traditional text representation methods.
02. Understand the concept of One-Hot Encoding.
03. Differentiate between sparse and dense vector representations.
04. Explain the concept of Word Embeddings.
05. Describe how Word2Vec learns word representations.
06. Compare CBOW and Skip-Gram architectures.
07. Implement Word2Vec using Python.

REVIEW

Review of Previous Lecture

TRADITIONAL FEATURE EXTRACTION

In the previous lecture, we studied:

- Bag of Words (BoW)
- Term Frequency (TF)
- Inverse Document Frequency (IDF)
- TF-IDF
- Cosine Similarity

These techniques convert text into numerical vectors that can be processed by machine learning algorithms.

QUESTION

Are these representations sufficient to understand the meaning of words?

ANSWER: **No.**

MOTIVATION

Why Do We Need Better Word Representation?

Traditional methods represent words using **frequency**, but they do not capture **meaning**.

Consider the following words:

Car

Automobile

Vehicle

Humans know these words are closely related. However, traditional representations treat them as completely independent.

THE PROBLEM

Machine learning models cannot recognize semantic relationships between words using BoW or TF-IDF alone.

MOTIVATION

Limitations of Traditional Methods

Bag of Words

- Ignores word meaning.
- Ignores semantic relationships.
- Produces high-dimensional sparse vectors.

TF-IDF

- Assigns better weights than BoW.
- Still cannot understand word meaning.
- Similar words are treated as different words.

EXAMPLE

King $\neq$ Queen

Car $\neq$ Automobile

Dog $\neq$ Puppy

Although these words are semantically related, traditional methods cannot recognize this relationship.

ONE-HOT ENCODING

One-Hot Encoding

DEFINITION

One-Hot Encoding is a technique used to represent each word as a binary vector.

Each word is assigned a unique position in the vocabulary.

Only one element in the vector is **1**, while all other elements are **0**.

KEY CHARACTERISTICS

- 01.* One word — One unique vector.
- 02.* Binary representation.
- 03.* Based on vocabulary index.

ONE-HOT ENCODING

Building a Vocabulary

Suppose we have the following vocabulary:

Index	Word
0	I
1	Love
2	NLP
3	AI
4	Python

Each word receives a unique index.

This index determines the position of **1** in its One-Hot vector.

ONE-HOT ENCODING

One-Hot Encoding Example

Using the previous vocabulary:

Word	One-Hot Vector
I	[1, 0, 0, 0, 0]
Love	[0, 1, 0, 0, 0]
NLP	[0, 0, 1, 0, 0]
AI	[0, 0, 0, 1, 0]
Python	[0, 0, 0, 0, 1]

OBSERVATION

- Every vector contains exactly **one value equal to 1**.
- All remaining values are **0**.

ONE-HOT ENCODING

Advantages & Limitations of One-Hot Encoding

ADVANTAGES

- Simple to understand.
- Easy to implement.
- Guarantees a unique representation for every word.

LIMITATIONS

- Very high-dimensional vectors.
- Sparse representation.
- No semantic information.
- Similar words appear completely unrelated.
- Vocabulary grows rapidly with new words.

EXAMPLE

King → [0,0,1,0,0]

Queen → [0,1,0,0,0]

Although semantically related, their vectors are completely different.

TRANSITION

From One-Hot Encoding to Word Embeddings

THE MAIN QUESTION

How can we represent words so that similar words have similar vectors?

Instead of using binary vectors, we represent each word using a **dense vector** of real numbers.

King → [0.82, -0.15, 0.63, 0.41, ...]

Queen → [0.80, -0.12, 0.60, 0.39, ...]

OBSERVATION

The vectors are different, but they are **mathematically close**, reflecting the similarity in meaning.

THIS IDEA LEADS TO...

Word Embeddings

WORD EMBEDDINGS

What are Word Embeddings?

DEFINITION

Word Embeddings are dense vector representations of words, where each word is represented by a vector of real numbers that captures its semantic and syntactic meaning.

Unlike One-Hot Encoding, Word Embeddings place **similar words close to each other** in the vector space.

KEY IDEA: *Words with similar meanings have similar vector representations.*

EXAMPLE

King → [0.82, -0.15, 0.63, 0.41]

Queen → [0.80, -0.12, 0.60, 0.39]

Man → [0.74, -0.10, 0.51, 0.32]

Woman → [0.72, -0.08, 0.49, 0.30]

Although the vectors are not identical, they are mathematically close because the words are semantically related.

WORD EMBEDDINGS

Sparse vs Dense Representation

SPARSE REPRESENTATION

EXAMPLES

One-Hot Encoding Bag of Words TF-IDF

CHARACTERISTICS

- High-dimensional vectors.
- Mostly zeros.
- Large memory usage.
- No semantic relationships.

DENSE REPRESENTATION

EXAMPLES

Word Embeddings Word2Vec GloVe FastText

CHARACTERISTICS

- Low-dimensional vectors.
- Real-valued numbers.
- Memory efficient.
- Preserves semantic relationships.

Sparse Vectors	Dense Vectors
Mostly 0s	Real numbers
High dimensions	Low dimensions
No meaning	Semantic meaning
Memory expensive	Memory efficient

WORD EMBEDDINGS

Why Are Word Embeddings Better?

TRADITIONAL REPRESENTATION

Only answers: "Does the word appear?"

Car
Automobile
Bus

All words are considered independent.

WORD EMBEDDINGS

Answers: "What does the word mean?"

Car — Automobile
|
Vehicle — Bus

Similar words become close in the vector space.

BENEFITS

Captures
semantic
similarity

Contextual
relationships

Improves ML
performance

WORD EMBEDDINGS

How Are Word Embeddings Learned?

Word Embeddings are **not manually assigned**.

Instead, they are **learned automatically** from a large collection of text.

BASIC IDEA

Words appearing in similar contexts tend to have similar meanings.

The cat is sleeping.

The dog is sleeping.

Since **cat** and **dog** appear in similar contexts, the model learns similar vector representations for them.

FAMOUS STATEMENT

"You shall know a word by the company it keeps."

— J. R. Firth (1957)

This principle is the foundation of most Word Embedding models.

WORD EMBEDDINGS

Applications of Word Embeddings

Word Embeddings are widely used in NLP tasks such as:

- Machine Translation

- Sentiment Analysis

- Text Classification

- Chatbots

- Question Answering

- Search Engines

- Recommendation Systems

- Named Entity Recognition

- Text Summarization

- *Because machines now understand semantic similarity, not just word frequency.*

WORD2VEC

Introduction to Word2Vec

DEFINITION

Word2Vec is one of the most popular algorithms for learning Word Embeddings.

It was introduced by **Google** in **2013**.

Instead of assigning random vectors, Word2Vec learns vector representations by analyzing the context in which words appear.

MAIN GOAL: Learn meaningful numerical representations of words from large text corpora.

TWO ARCHITECTURES

Continuous Bag of Words

CBOW

Skip-Gram

Skip-Gram Model

Both produce high-quality embeddings.

WORD2VEC

Continuous Bag of Words (CBOW)

DEFINITION

Continuous Bag of Words (CBOW) is a Word2Vec architecture that predicts the **target word** using its surrounding context words.

Context → Target Word

CHARACTERISTICS

- Uses surrounding words as input.
- Predicts the center (target) word.
- Faster to train.
- Performs well with frequent words.

EXAMPLE

Sentence:

I love Natural Language
Processing

Target word:

love

Input (Context):

I Natural Language

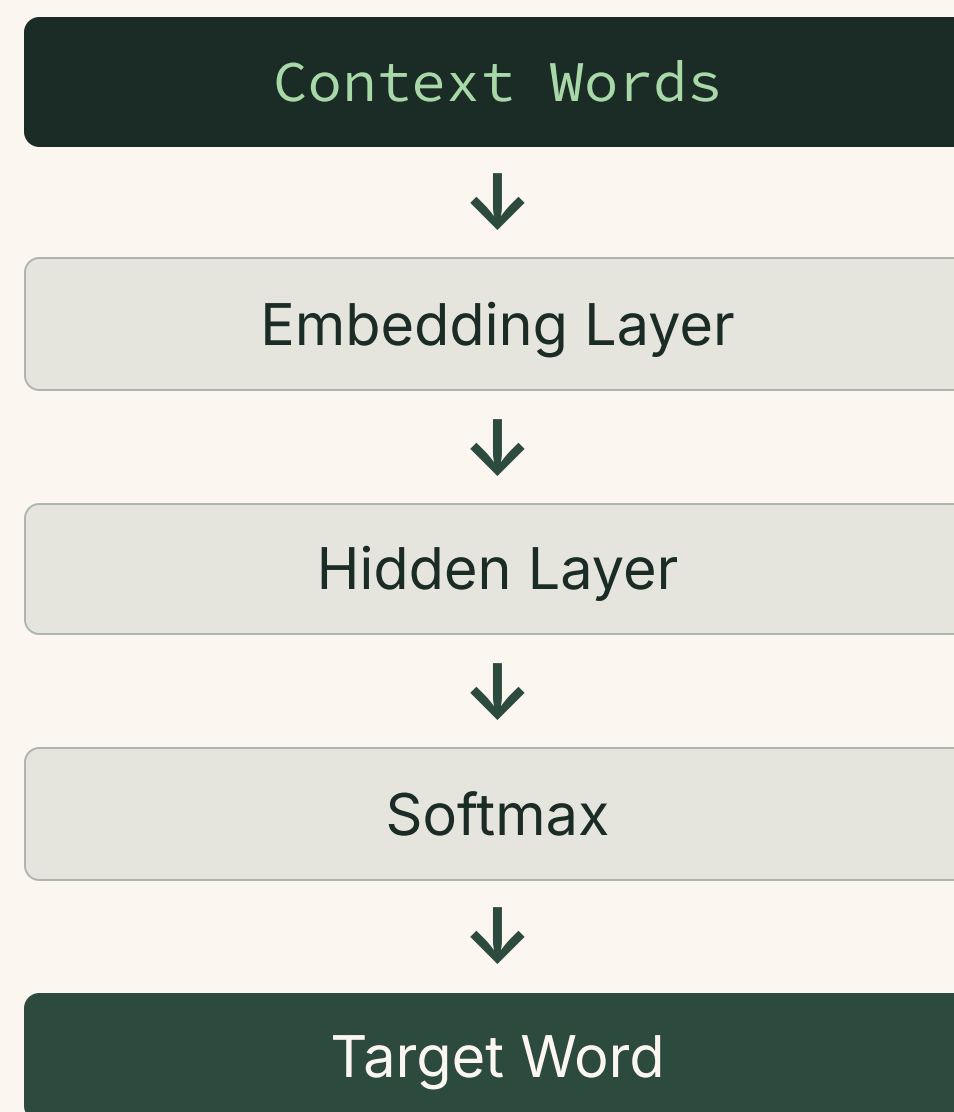
Output:

love

WORD2VEC

CBOW Architecture

WORKFLOW



EXAMPLE

Sentence:

The cat sits on the mat

INPUT

The
sits

OUTPUT

cat

The model learns that "The" and "sits" often appear around "cat", helping it learn meaningful word representations.

WORD2VEC

Skip-Gram Model

DEFINITION

Skip-Gram is the second architecture of Word2Vec.

Instead of predicting the target word from its context, Skip-Gram predicts the **surrounding context words** using the target word.

Target Word → Context Words

CHARACTERISTICS

- Uses one target word as input.
- Predicts multiple surrounding words.
- Better for learning rare words.
- Usually requires more training time than CBOW.

EXAMPLE

Sentence:

I love Natural Language
Processing

Target Word:

love

Predicted Context:

I
Natural
Language

WORD2VEC

CBOW vs Skip-Gram

CBOW	Skip-Gram
Predicts the target word	Predicts surrounding words
Input: Context words	Input: Target word
Output: Target word	Output: Context words
Faster training	Slower training
Better for frequent words	Better for rare words
Simpler model	Learns richer representations

CBOW — SENTENCE: "I LOVE NLP"

Input: I, NLP → Output: love
[I, NLP] → love

SKIP-GRAM — SENTENCE: "I LOVE NLP"

Input: love → Output: I, NLP
love → [I, NLP]

WORD2VEC

Advantages and Limitations of Word2Vec

ADVANTAGES

- Learns semantic relationships between words.
- Produces dense vector representations.
- Reduces dimensionality compared with One-Hot Encoding.
- Improves the performance of many NLP applications.
- Efficient when trained on large text corpora.

LIMITATIONS

- Cannot represent words not seen during training (Out-of-Vocabulary).
- Assigns only **one vector per word**, even if the word has multiple meanings.
- Requires a large amount of training data.

EXAMPLE

The word "**Bank**" can mean a financial institution or a river bank. Word2Vec assigns **one vector** to both meanings, which may cause ambiguity.

GLOVE & FASTTEXT

GloVe (Global Vectors)

DEFINITION

GloVe (Global Vectors for Word Representation) is a word embedding algorithm developed by researchers at **Stanford University** in 2014.

Unlike Word2Vec, GloVe learns word representations by analyzing **global word co-occurrence statistics** across the entire corpus.

KEY IDEA: GloVe considers how often words appear together in the **whole dataset**.

EXAMPLE

Corpus:

King and Queen

King is a man

Queen is a woman

King rules a kingdom

Queen rules a kingdom

*The model observes that **King** and **Queen** frequently appear in similar contexts. Therefore, their vector representations become similar.*

GLOVE & FASTTEXT

Word2Vec vs GloVe

Word2Vec	GloVe
Learns from local context	Learns from global co-occurrence statistics
Predictive model	Count-based model
Processes one context window at a time	Uses the entire corpus
Excellent semantic representations	Excellent semantic representations
Efficient for large datasets	Captures global relationships more effectively

ADVANTAGES OF GLOVE

- Captures both local and global information.
- Produces meaningful word vectors.
- Efficient for large corpora.
- Widely used in NLP research.

LIMITATION

Like Word2Vec, GloVe assigns **only one vector for each word**, regardless of its different meanings.

GLOVE & FASTTEXT

FastText

DEFINITION

FastText is a word embedding model developed by **Facebook AI Research (FAIR)**.

Unlike Word2Vec and GloVe, FastText represents each word as a collection of **character n-grams** instead of treating the word as a single unit.

KEY IDEA: A word is built from smaller character sequences, enabling the model to learn information about the **internal structure** of words.

EXAMPLE

Word:

Learning

Character n-grams:

Le Lea Lear
earn rnin ing
...

The final word vector is computed from these character n-grams.

Advantages of FastText

ADVANTAGES

- Handles rare words effectively.
- Can generate vectors for unseen words.
- Captures morphological information.
- Works well with languages that have rich word formation.

LIMITATION

- Requires more memory than Word2Vec.
- Slightly slower during training due to processing character n-grams.

EXAMPLE

Suppose the model has learned:

Play
Player
Playing

Even if it has never seen:

Playable

FastText can still generate a meaningful vector because it understands the shared character patterns.

GLOVE & FASTTEXT

Comparison of Word Embedding Models

Feature	Word2Vec	GloVe	FastText
Uses Context	Yes	Yes	Yes
Uses Global Statistics	No	Yes	No
Uses Character n-grams	No	No	Yes
Handles Rare Words	Limited	Limited	Excellent
Handles Unseen Words	No	No	Yes
Training Speed	Fast	Fast	Moderate

WHICH MODEL SHOULD WE USE?

Word2Vec — Efficient and widely used.

GloVe — Better at capturing global relationships.

FastText — Best when handling rare words, misspellings, or morphologically rich languages.

PYTHON LAB

Python Lab: Word2Vec

OBJECTIVE

In this lab, we will learn how to:

- Train a Word2Vec model.
- Generate Word Embeddings.
- Find similar words.
- Perform word analogy.
- Explore vector representations.

REQUIRED LIBRARIES

```
from gensim.models import Word2Vec
```

SAMPLE DATASET

```
sentences = [  
    ["i", "love", "nlp"],  
    ["i", "love", "machine", "learning"],  
    ["artificial", "intelligence", "uses",  
    "nlp"],  
    ["machine", "learning", "is",  
    "interesting"],  
    ["deep", "learning", "is", "part",  
    "of", "ai"]  
]
```

Training a Word2Vec Model

```
from gensim.models import Word2Vec  
  
model = Word2Vec(  
    sentences,  
    vector_size=100,  
    window=2,  
    min_count=1,  
    workers=4  
)
```

PARAMETERS

Parameter	Description
vector_size	Dimension of the embedding vector
window	Number of surrounding words (context window)
min_count	Minimum frequency of a word to be included
workers	Number of CPU threads used during training

OBSERVATION

After training, every word in the vocabulary has its own dense vector representation.

Finding Similar Words

Once the model is trained, we can search for words with similar meanings.

```
model.wv.most_similar("learning")
```

INTERPRETATION

The model places semantically related words close together in the vector space.

The similarity score ranges from **0 to 1**, where values closer to **1** indicate greater similarity.

EXAMPLE OUTPUT

word	similarity
machine	0.89
deep	0.82
nlp	0.76

Word Analogy

One of the most impressive capabilities of Word2Vec is solving word analogy problems.

```
model.wv.most_similar(  
    positive=["king", "woman"],  
    negative=["man"],  
    topn=1  
)
```

INTERPRETATION

The model learns semantic relationships between words. It understands that:

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$$

EXAMPLE OUTPUT

RESULT

Queen

This demonstrates that word vectors capture meaningful linguistic patterns.

SUMMARY

Chapter Summary

In this lecture, we learned:

- Limitations of traditional text representations.
- One-Hot Encoding.
- Sparse vs Dense vectors.
- Word Embeddings.
- Word2Vec.
- CBOW.
- Skip-Gram.
- GloVe.
- FastText.
- Word Embeddings using Python.

EVOLUTION OF WORD REPRESENTATION

One-Hot Encoding



Word Embeddings



Word2Vec (CBOW / Skip-Gram)



GloVe



FastText

Representations become more expressive as we progress.

SUMMARY

Key Takeaways

- 01.* One-Hot Encoding provides a unique representation but does not capture meaning.
- 02.* Word Embeddings represent words as dense numerical vectors.
- 03.* Similar words have similar vector representations.
- 04.* Word2Vec learns embeddings from surrounding context.
- 05.* CBOW predicts the target word from its context.
- 06.* Skip-Gram predicts context words from the target word.
- 07.* GloVe incorporates global co-occurrence statistics.
- 08.* FastText represents words using character n-grams, enabling it to handle unseen and rare words more effectively.

ASSESSMENT

Quiz

Q1.

What is the main limitation of One-Hot Encoding?

- A) Cannot represent numbers
- B) Sparse vectors without semantic meaning**
- C) Requires deep learning
- D) Cannot represent documents

Q2.

Which model predicts the target word from surrounding words?

- A) Skip-Gram
- B) CBOW**
- C) GloVe
- D) FastText

Q3.

Which Word2Vec architecture predicts surrounding words using the target word?

- A) CBOW
- B) TF-IDF
- C) Skip-Gram**
- D) Bag of Words

Q4.

Which model uses global word co-occurrence statistics?

- A) Word2Vec
- B) GloVe**
- C) FastText
- D) One-Hot Encoding

Q5.

Which model represents words using character n-grams?

- A) Word2Vec
- B) GloVe
- C) FastText**
- D) TF-IDF

ANSWER KEY

1 – B

2 – B

3 – C

4 – B

5 – C



Thank You

Questions?

NEXT LECTURE

Sequence Models

Recurrent Neural Networks (RNN), LSTM & GRU